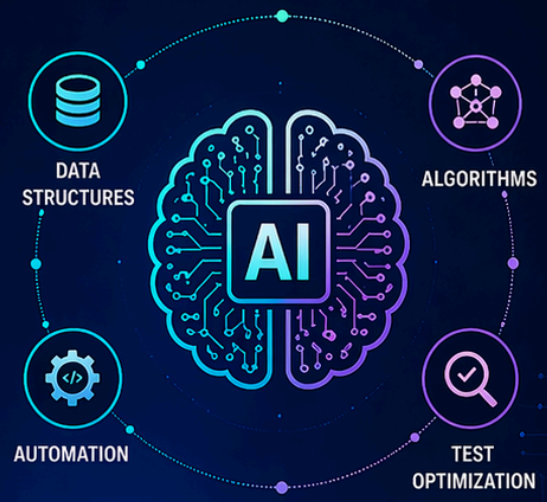
WHITEPAPER

DSA FOR SDETs

IN THE AGE OF AI-AUGMENTED TESTING



Leveraging Data Structures & Algorithms to build **intelligent**, **scalable**, and **optimized** testing solutions.

Why algorithmic thinking has become the leverage layer between AI-generated test volume and reliable engineering outcomes.

HASHING
Fast lookups, duplicate detection & data validation

SLIDING WINDOW
Analyze logs, metrics & detect anomalies in real time

GRAPH TRAVERSAL
Validate dependencies, workflows & state transitions

HEAPS / PRIORITY QUEUES
Prioritize tests, schedule execution & manage flaky tests

DP
DYNAMIC PROGRAMMING
Optimize computations, reuse results & improve performance

AUTHOR

SDET INSIGHTS

Kavita Jadhav
Quality Engineering Leader
Test Automation Architect

PUBLISHED
May 25, 2025

K11 SOFTWARE SOLUTIONS LLC
Engineering **Quality**. Delivering **Excellence**.

DSA for SDETs in the Age of AI-Augmented Testing

Why algorithmic thinking has become the leverage layer between AI-generated test volume and reliable engineering outcomes.

Author: Kavita Jadhav
Organization: K11 Software Solutions LLC
Published: April 22, 2026

Executive summary

AI-augmented testing is no longer a niche practice. GitHub's official Copilot documentation now teaches engineers how to generate unit and integration tests, while Playwright ships both a test generator and agent workflows that can plan, generate, and heal tests. At the organizational level, Google Cloud's 2025 DORA research reports that almost 90% of respondents are already using AI, but says the best returns come from foundational systems and strong engineering capabilities rather than tools alone. [1]

For SDETs, that changes the center of gravity of the job. The hard part is increasingly not "writing one more test," but deciding which AI-generated tests to keep, how to remove duplicates, how to rank critical paths under CI time pressure, and how to detect unusual failure patterns early. That shift is visible in the data: Sonar's 2026 enterprise developer survey found that 96% of developers do not fully trust AI output and only 48% always verify it before committing, while GitHub's own testing guidance says Copilot does best on simpler cases and needs more detailed prompts and validation for complex scenarios. [2]

That is exactly where DSA matters. Hashing gives fast deduplication and stable fingerprints; priority queues convert limited time budgets into risk-first execution; deques make streaming windows cheap; graph traversal makes dependency-aware testing practical; and range, prefix, and clustering patterns turn noisy telemetry into usable signal. In other words, AI helps create candidate tests, but DSA helps an SDET turn those candidates into an efficient, explainable, production-grade quality system. [3]

Using DSA with AI

The first reason is that AI changes output volume faster than it changes verification discipline. GitHub Copilot can help write tests, and Playwright can generate tests from browser interactions or run planner-generator-healer loops, but neither capability

removes the need to verify correctness, isolate tests, and keep suites maintainable. Playwright's own best-practices guide still emphasizes testing user-visible behavior and making tests isolated and reproducible. [4]

The second reason is that AI-generated tests are better than they were a year ago, but they are still imperfect enough to need strong post-processing. FSE 2024's ChatTester reported 34.3% more compilable tests and 18.7% more tests with correct assertions than default ChatGPT by adding an iterative refiner. ICSE-SEIP 2025's ASTER went further: on Python applications it improved line coverage from 44.0% to 78.0%, branch coverage from 53.7% to 77.2%, and method coverage from 61.2% to 86.7%; in a survey of 161 professionals, 70% of Java ASTER tests and 88% of Python ASTER tests would be added to a regression suite with no or only minor changes. Those are strong results, but they also underline the point: good AI testing pipelines are hybrid systems that combine generation, repair, filtering, and ranking. [5]

The third reason is that CI remains a time-constrained optimization problem. Recent research on continuous-integration test-case prioritization shows that contextual information, business data, test characteristics, and historical execution behavior can be used to rank tests so that the most informative ones run sooner. Google Cloud's DORA work makes the same broader point from an organizational angle: AI is an amplifier, not a substitute for engineering foundations. For experienced SDETs, DSA is one of those foundations because it converts an AI-heavy workflow into deterministic rules for selection, ordering, and anomaly detection. [6]

Top patterns every SDET should keep handy

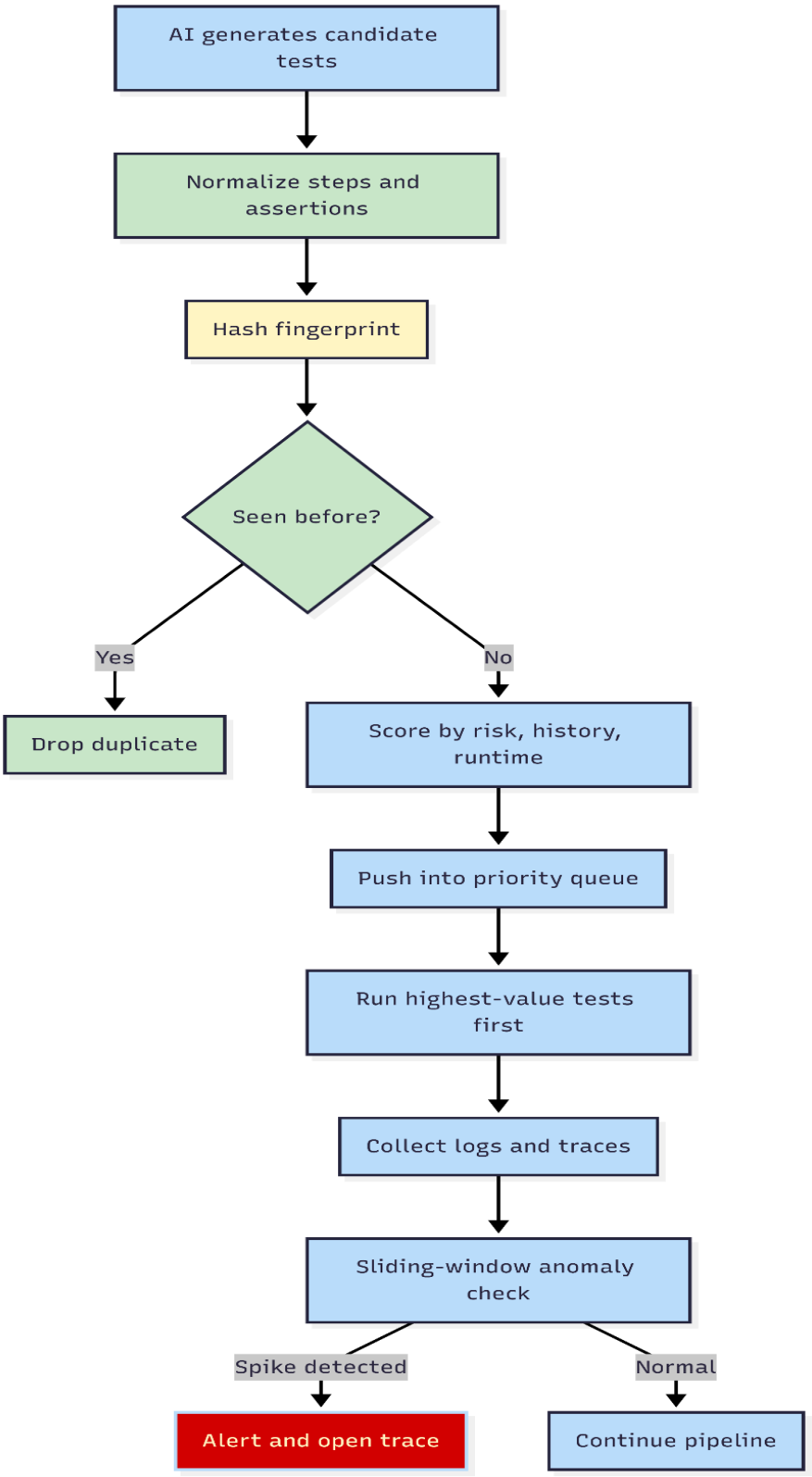
The most transferable DSA patterns for AI-augmented testing are the ones that solve curation, prioritization, observability, and dependency problems repeatedly seen in official tooling, recent test-generation research, and CI-prioritization literature. [7]

- **Hashing and sets.** Canonicalize prompts or generated tests, compute a fingerprint, and drop duplicates before they burn CI minutes. [8]
- **Heap and priority queue.** Run high-risk, high-value, or recently flaky tests first when the pipeline is time-boxed. [9]
- **Sliding window.** Track the last *N* failures, log events, or latency samples and alert when the local pattern changes sharply. [10]
- **Graph traversal.** Walk user journeys, service dependencies, or impact maps when one code change can affect many paths. [11]
- **Interval merge.** Combine overlapping maintenance windows, retries, trace segments, or flaky-event spans into clearer failure timelines. [12]
- **Top-K selection.** Surface the slowest tests, noisiest failures, or most suspicious AI-generated cases without scanning every dashboard manually. [13]

- **Union-find and clustering.** Group tests that fail together because of shared infrastructure, shared fixtures, or shared dependency edges. This is especially useful in large suites with systemic flake patterns. The clustering idea follows naturally from CI-prioritization research that exploits historical relationships and contextual signals. [14]
- **Binary search and bisecting.** Narrow failing commits, flag thresholds, timeout budgets, or configuration regressions by halving the search space instead of linearly guessing. This remains one of the fastest ways to isolate root cause in noisy AI-heavy pipelines. [15]

Practical recipes for an AI-augmented test pipeline

A useful mental model is to treat AI as a generator of candidate tests and DSA as the control plane that keeps those candidates valuable. The flow below is the simplest version of that loop. It maps cleanly to the kinds of generation, healing, refinement, verification, and CI-time decision-making described in Playwright, GitHub Copilot, Sonar, and recent test-generation research. [16]


AI generates candidate tests
Normalize steps and assertions
Hash fingerprint
Seen before?
Yes
No
Drop duplicate
Score by risk, history, runtime
Push into priority queue
Run highest-value tests first
Collect logs and traces
Sliding-window anomaly check
Spike detected
Normal
Alert and open trace
Continue pipeline

Deduping AI-generated tests with hashing

```
import hashlib
import json
import re
from typing import Iterable

def canonicalize(test_case: dict) -> str:
    """Normalize AI-generated tests so cosmetic differences hash the same."""
    steps = [re.sub(r"\s+", " ", s.strip().lower()) for s in test_case["steps"]]
          assertions  =  [re.sub(r"\s+",  "  ",  a.strip().lower())  for  a  in
test_case["assertions"]]
    payload = {
        "feature": test_case.get("feature", "").strip().lower(),
        "steps": steps,
        "assertions": sorted(assertions),
    }
    return json.dumps(payload, sort_keys=True)

def dedupe_tests(candidates: Iterable[dict]) -> list[dict]:
    seen = set()
    unique = []

    for tc in candidates:
                            digest   =   hashlib.blake2b(canonicalize(tc).encode(),
digest_size=16).hexdigest()
        if digest in seen:
            continue
        seen.add(digest)
        unique.append({**tc, "fingerprint": digest})

    return unique
```

This is the fastest guardrail to add when an LLM starts overproducing slight rewrites of the same scenario. Python's documentation explicitly describes sets as useful for removing duplicate entries and fast membership testing, which is exactly the access pattern SDETs need here. In practice, this step is most valuable immediately after AI generation and before expensive execution or human review. [8]

Prioritizing tests with a heap-based run queue

```python
import heapq
from dataclasses import dataclass, field

@dataclass(order=True)
class PrioritizedTest:
    priority: float
    name: str = field(compare=False)
    est_runtime_sec: int = field(compare=False, default=60)
    failure_rate_30d: float = field(compare=False, default=0.0)
    changed_area_overlap: float = field(compare=False, default=0.0)
    business_criticality: int = field(compare=False, default=1)

def score(test: PrioritizedTest) -> float:
    # Lower value runs first because heapq is a min-heap.
    risk = (
        3.0 * test.failure_rate_30d +
        2.5 * test.changed_area_overlap +
        1.5 * test.business_criticality -
        0.01 * test.est_runtime_sec
    )
    return -risk

def build_run_queue(tests: list[PrioritizedTest]) -> list[PrioritizedTest]:
    heap = []
    for t in tests:
        t.priority = score(t)
        heapq.heappush(heap, t)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Python's `heapq` module is the standard implementation of a priority queue, and CI prioritization research shows why that matters: modern pipelines are time-constrained, and recent work has shown that contextual information and historical features can improve how teams rank the tests most likely to reveal faults early. A heap gives you a simple, explainable way to operationalize that logic before you ever reach for a more complex ML model. [9]

Sliding-window anomaly detection on logs

```
from collections import deque
from statistics import mean

class ErrorWindow:
    def __init__(self, size: int = 20, threshold: float = 2.0):
        self.window = deque(maxlen=size)
        self.threshold = threshold

    def update(self, error_count_last_minute: int):
        self.window.append(error_count_last_minute)
        if len(self.window) < self.window.maxlen:
            return {"anomaly": False, "baseline": None}

        baseline = mean(list(self.window)[:-1])  # compare latest point to recent
history
        latest = self.window[-1]

        return {
            "anomaly": latest > max(self.threshold * baseline, baseline + 5),
            "baseline": round(baseline, 2),
            "latest": latest,
        }

# Example use while streaming CI logs:
detector = ErrorWindow(size=15, threshold=1.8)
for count in [2, 1, 3, 2, 2, 3, 2, 1, 2, 3, 2, 2, 3, 2, 11]:
    result = detector.update(count)
    if result["baseline"] is not None and result["anomaly"]:
        print("Alert:", result)
```

Python's `deque` is particularly well suited to this pattern because it supports memory-efficient appends and pops from either side with approximately O(1) performance, and its bounded `maxlen` form is explicitly meant for tracking only the most recent activity. That makes it a natural fit for live test logs, retry bursts, API error spikes, or sudden increases in flaky failures. When a spike occurs, Playwright Trace Viewer gives SDETs a fast way to inspect a failed run visually in CI. [17]

A short case study

The clearest public illustration of the opportunity comes from Duolingo's 2025 engineering write-up on using GPT-based regression tooling. Their QA team described a workflow in which natural-language instructions were used to automate previously difficult-to-automate regression paths, shifting the team from manual walkthroughs to reviewing recorded runs; the result was a reduction in manual regression workflows of up to 70%. That is a major outcome on its own, but it also points directly to the next problem SDETs face once AI starts helping: how to keep the new stream of generated scenarios trustworthy, non-duplicative, and priority-aware. [18]

That is where the recent research lines up with a DSA-centric operating model. FSE 2024's ChatTester showed that adding an iterative refinement loop to baseline ChatGPT produced 34.3% more compilable tests and 18.7% more tests with correct assertions. ICSE-SEIP 2025's ASTER then showed that an LLM-plus-program-analysis pipeline can materially improve both coverage and maintainability: on Python applications, ASTER increased line coverage from 44.0% to 78.0%, and 88% of surveyed developers said its Python tests could enter regression suites with no or only minor changes. In other words, the path to measurable impact is no longer "AI or DSA." It is AI for generation, plus DSA for deduplication, ranking, and signal extraction. That combination is what turns test quantity into test utility. [5]

Recommended learning path and resources

For experienced SDETs, the best learning path is not to relearn all of computer science in the abstract. It is to study the specific DSA concepts that recur in production testing systems: collections for fast membership, queues for ordering, windows for streams, and graph-like thinking for impact and journey coverage. The official and primary sources below are ordered for that use case. [19]

Start with **day-to-day DSA books**. *A Common-Sense Guide to Data Structures and Algorithms* emphasizes practical scenarios, practice exercises, and everyday performance tradeoffs; *Grokking Algorithms* is a strong visual refresher for programmers who want approachable examples in Python; and *Introduction to Algorithms* remains the most comprehensive reference when your team needs depth on design techniques, proofs, and advanced structures. [20]

Add **two primary courses** rather than ten secondary ones. MIT OpenCourseWare's *Introduction to Algorithms* is still one of the strongest free foundations for algorithmic modeling and performance analysis, while Princeton's *Algorithms, Part I* and *Part II* are especially practical for data structures, sorting, search, priority queues, graphs, and string algorithms. [21]

For **AI-augmented testing practice**, go straight to the official docs and current research. GitHub's Copilot testing guide is the right source for prompt strategy and expectations; Playwright's Test Agents, code generation, best-practice, and Trace Viewer docs are immediately actionable for modern web testing; Google Cloud's 2025 DORA reports are useful for the organizational view of AI adoption; Sonar's survey and AI Code Assurance guidance are strong references for governance and verification; and ASTER plus ChatTester are the most relevant recent research sources for test-generation quality and acceptability. [22]

For **tools**, the most useful stack for Python-leaning SDETs is: Python's official docs for `set`, `heapq`, and `deque`; `pytest` for execution and fixtures; Playwright for browser automation and traces; GitHub Actions or your CI equivalent for sharding and orchestration; and SonarQube AI Code Assurance or similar static-analysis gates to verify AI-generated code and tests before they land. [23]

The five next steps I would recommend to SDETs at K11 are straightforward:

1. **Add a dedupe layer in front of AI-generated tests.** If your team is using Copilot, prompt files, or browser-generated scenarios, hash normalized steps and assertions before execution. [24]
2. **Replace FIFO test ordering with risk-based priority queues.** Even a simple historical-failure-and-runtime score will usually outperform "run everything in collection order." [9]
3. **Instrument logs and traces with bounded windows.** A small deque-backed detector will catch local spikes faster than end-of-run summary reports. [17]
4. **Treat AI-generated tests as candidates, not truth.** Put quality gates, review steps, and trace-based debugging around them by default. [25]
5. **Train the team on five patterns first:** hashing, heap/priority queue, sliding window, graph traversal, and interval merge. That short list covers a surprisingly large share of AI-augmented testing work. The recommendation follows directly from the tooling and research cited throughout this newsletter. [26]

---

[1] [4] [7] [16] [22] [24] [27] Writing tests with GitHub Copilot - GitHub Docs

https://docs.github.com/en/copilot/tutorials/write-tests

[2] [28] State of Code Developer Survey report | Sonar

https://www.sonarsource.com/resources/developer-survey-report/

[3] [23] [29] 3. Data model — Python 3.14.3 documentation

https://docs.python.org/3/reference/datamodel.html?utm_source=chatgpt.com

[5] Evaluating and Improving ChatGPT for Unit Test Generation (FSE 2024 - Research Papers) - FSE 2024

https://2024.esec-fse.org/details/fse-2024-research-papers/34/Evaluating-and-Improving-ChatGPT-for-Unit-Test-Generation

[6] Comparative study of machine learning test case prioritization for continuous integration testing | Software Quality Journal | Springer Nature Link

https://link.springer.com/article/10.1007/s11219-023-09646-0

[8] [19] [32] 5. Data Structures — Python 3.14.3 documentation

https://docs.python.org/3/tutorial/datastructures.html?utm_source=chatgpt.com

[9] [13] heapq — Heap queue algorithm — Python 3.14.4 documentation

https://docs.python.org/3/library/heapq.html

[10] [17] collections — Container datatypes — Python 3.14.4 documentation

https://docs.python.org/3/library/collections.html

[11] [26] Agents | Playwright

https://playwright.dev/docs/test-agents

[12] Trace viewer | Playwright

https://playwright.dev/docs/trace-viewer

[14] On the use of contextual information for machine learning based test case prioritization in continuous integration development - ScienceDirect

https://www.sciencedirect.com/science/article/pii/S0950584924000491

[15] Best Practices | Playwright

https://playwright.dev/docs/best-practices

[18] [30] Duolingo Blog

https://blog.duolingo.com/reduced-regression-testing/

[20] A Common-Sense Guide to Data Structures and Algorithms, Second Edition: Level Up Your Core Programming Skills by Jay Wengrow

https://pragprog.com/titles/jwdsal2/a-common-sense-guide-to-data-structures-and-algorithms-second-edition/

[21] [31] Introduction to Algorithms | Electrical Engineering and Computer Science | MIT OpenCourseWare

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[25] AI Code Assurance: Secure Quality AI-Generated Code Development | Sonar

https://www.sonarsource.com/solutions/ai/ai-code-assurance/